REAL TIME GROUNDWATER RESOURCE EVALUATION USING DWLR DATA

Daily Work Report & Backend Development Theory

Coverage Period: June 21 – June 30

Module: Backend Development (Spring Boot) & Full-Stack Integration

June 21 — Database Connectivity

Activity: On June 21, the Spring Boot backend project is connected to the database for the Real Time Groundwater Resource Evaluation System. This establishes the live link between the application code and the persistent storage where DWLR station and groundwater reading data will reside.

What is Database Connectivity?

Database Connectivity refers to configuring and verifying the link between the Spring Boot application and the relational database (MySQL/PostgreSQL) that stores groundwater monitoring data. Since DWLR stations continuously generate time-series readings, the connection must be stable, secure, and capable of handling frequent read/write operations without performance degradation.

This step also involves designing the initial schema for core tables — DWLR Station, Groundwater Reading, and User — and confirming that Spring Data JPA can map Java entities to these tables correctly through Hibernate.

Key Tasks Involved:

- Configure the datasource URL, username, password, and driver class in application.properties/application.yml.
- Select and configure the Hibernate dialect appropriate to the chosen database (MySQL/PostgreSQL).
- Set the Hibernate DDL-auto property (validate/update) appropriately for the development environment.
- Create the groundwater_monitoring database schema and verify it is reachable from the local/development environment.
- Test the connection using a basic health-check endpoint or Spring Boot Actuator's /actuator/health.
- Configure connection pooling (HikariCP) parameters such as maximum pool size and idle timeout for stable performance.
- Set up a database migration tool (Flyway/Liquibase) to version-control schema changes going forward.
- Verify that the application starts without errors and logs a successful database connection on boot.

Expected Deliverable: **DB Connection —** A verified, stable connection between the Spring Boot backend and the relational database, with the groundwater monitoring schema reachable and ready for entity mapping.

Technology & Purpose

Technology	Purpose
Spring Boot	Core backend framework managing the datasource configuration
Spring Data JPA	ORM layer bridging Java entities and database tables
MySQL / PostgreSQL	Relational database storing DWLR station and reading records
HikariCP	High-performance connection pooling for database access
Flyway / Liquibase	Version-controlled database schema migrations
Spring Boot Actuator	Health-check endpoint to verify DB connectivity

June 22 — Entity & Repository Creation

Activity: On June 22, the JPA Entity classes and Spring Data Repository interfaces are created for the core domain objects of the Real Time Groundwater Resource Evaluation System. This forms the persistence layer through which all groundwater data will be read from and written to the database.

What is Entity & Repository Creation?

Entity creation involves defining Java classes annotated with JPA annotations (@Entity, @Table, @Id) that map directly to database tables such as DwlrStation, GroundwaterReading, and User. Each entity represents one row of data and its relationships to other entities — for example, a DWLR Station has many Groundwater Readings.

Repository creation involves defining Spring Data JPA repository interfaces that extend JpaRepository, giving the application built-in methods for common database operations (save, findById, findAll, delete) without writing manual SQL, along with custom query methods for groundwater-specific lookups.

Key Tasks Involved:

- Define the DwlrStation entity with fields: stationId, location, latitude, longitude, district, block, installationDate, status.
- Define the GroundwaterReading entity with fields: readingId, stationId (foreign key), waterLevel, rainfall, batteryStatus, recordedAt.
- Define the User entity with fields: userId, name, email, passwordHash, role (Admin/Field Engineer/Viewer).
- Establish entity relationships — One-to-Many between DwlrStation and GroundwaterReading using @OneToMany/@ManyToOne.
- Create repository interfaces (DwlrStationRepository, GroundwaterReadingRepository, UserRepository) extending JpaRepository.
- Add custom query methods — e.g., findByDistrict(), findByStationIdAndDateBetween(), findByStatus().
- Apply Lombok annotations (@Data, @Builder, @NoArgsConstructor) to reduce boilerplate in entity classes.
- Verify entity-to-table mapping by running the application and confirming the schema matches expectations (or via Flyway migration scripts).

Expected Deliverable: **Entity Classes —** A complete set of JPA entity classes and corresponding Spring Data repository interfaces, correctly mapped to the database schema and ready to support CRUD and query operations.

Technology & Purpose

Technology	Purpose
Spring Data JPA	Defining entities and repository interfaces
Hibernate	ORM engine mapping entities to database tables
Lombok	Reducing boilerplate code (getters, setters, builders) in entities
MySQL / PostgreSQL	Underlying tables that entities map to

Spring Boot	Dependency injection of repositories into service layer

June 23 — REST API Development

Activity: On June 23, the REST API endpoints for core CRUD operations are developed for the Real Time Groundwater Resource Evaluation System. These APIs expose DWLR station and groundwater reading data to the React frontend over HTTP.

What is REST API Development?

REST API Development involves building Controller classes that expose HTTP endpoints (GET, POST, PUT, DELETE) for each core entity, following REST conventions for resource naming and status codes. Each endpoint delegates business logic to the Service layer, which in turn uses the Repository layer to interact with the database.

For a groundwater monitoring system, the APIs must support efficient retrieval of large volumes of time-series sensor data, as well as standard create/update/delete operations for station and user management.

Key Tasks Involved:

- Create DwlrStationController with endpoints: GET /api/stations, GET /api/stations/{id}, POST /api/stations, PUT /api/stations/{id}, DELETE /api/stations/{id}.
- Create GroundwaterReadingController with endpoints for adding new readings and retrieving readings by station, date range, or district.
- Implement DTO (Data Transfer Object) classes to control exactly what data is exposed in API responses, separate from entity classes.
- Apply request validation using @Valid and Bean Validation annotations (@NotNull, @Min, @Max) on incoming DTOs.
- Standardize API responses using a common wrapper class (e.g., ApiResponse) including status, message, and data fields.
- Implement global exception handling using @ControllerAdvice to return consistent error responses.
- Add pagination and sorting support to list endpoints using Spring Data's Pageable.
- Document all endpoints (request/response formats) for frontend integration and Postman testing.

Expected Deliverable: CRUD APIs — A complete set of tested REST API endpoints supporting Create, Read, Update, and Delete operations for DWLR stations and groundwater readings, ready for frontend integration.

Technology & Purpose

Technology	Purpose
Spring Boot (Web)	Building REST controllers and routing HTTP requests
Spring Data JPA	Persisting and retrieving data through the service layer
Lombok	Reducing boilerplate in DTOs and entities
Bean Validation (javax/jakarta.validation)	Validating incoming request data

Jackson	Serializing/deserializing JSON request and response bodies
Postman	Manual verification of endpoints during development

June 24 — Authentication Module

Activity: On June 24, the authentication module is developed for the Real Time Groundwater Resource Evaluation System, enabling secure login for Admin, Field Engineer, and Viewer roles using token-based authentication.

What is the Authentication Module?

The Authentication Module is responsible for verifying user identity at login and issuing a secure token (JWT) that the frontend attaches to subsequent API requests. Since the system serves multiple roles with different access levels, authentication must work hand-in-hand with role-based authorization to protect sensitive groundwater data and administrative functions.

This module also covers password security, ensuring that credentials are never stored or transmitted in plain text, and that login attempts are validated against properly hashed passwords.

Key Tasks Involved:

- Implement the /api/auth/login endpoint that validates email/username and password against stored (hashed) credentials.
- Configure Spring Security to use BCryptPasswordEncoder for hashing and verifying passwords.
- Generate JSON Web Tokens (JWT) on successful login, embedding user role and expiry claims.
- Implement a JWT filter (OncePerRequestFilter) to validate the token on every incoming request and set the security context.
- Configure role-based access rules (Admin, Field Engineer, Viewer) using Spring Security's authorizeHttpRequests/antMatchers.
- Implement the /api/auth/register endpoint with validation to prevent duplicate email registration.
- Add refresh-token or token-expiry handling so sessions can be renewed without forcing repeated logins.
- Return clear, standardized error responses for invalid credentials, expired tokens, and unauthorized access attempts.

Expected Deliverable: **Login API —** A secure, fully functional authentication module issuing JWT tokens on login, with role-based access control enforced across protected API endpoints.

Technology & Purpose

Technology	Purpose
Spring Security	Core framework for authentication and authorization
JWT (jjwt / java-jwt)	Issuing and validating stateless authentication tokens
BCryptPasswordEncoder	Secure one-way hashing of user passwords
Spring Boot	Wiring security configuration into the application context

Spring Data JPA	Looking up user records during authentication

June 25 — Backend Business Logic

Activity: On June 25, the core business logic for the Real Time Groundwater Resource Evaluation System is implemented in the Service layer, going beyond basic CRUD to handle groundwater-specific rules and calculations.

What is Backend Business Logic?

Backend Business Logic refers to the rules, calculations, and workflows that give meaning to raw groundwater data — for example, determining whether a station's water level falls into a Normal, Warning, or Critical category, computing trends over time, or flagging stations that have stopped reporting data (possible sensor failure).

This logic lives in the Service layer, sitting between the Controller (which handles HTTP requests) and the Repository (which handles data access), keeping the codebase clean, testable, and reusable across multiple API endpoints.

Key Tasks Involved:

- Implement threshold-based classification logic to tag each reading as Normal, Warning, or Critical based on configurable water-level limits.
- Implement aggregation logic to compute average, minimum, and maximum groundwater levels per station, district, or block.
- Implement trend-detection logic comparing recent readings against historical averages to flag rising or falling groundwater levels.
- Implement stale-data detection to flag DWLR stations that have not reported readings within an expected interval.
- Apply business rules for role-based data visibility (e.g., Field Engineers only see their assigned stations) within the service layer.
- Implement transactional boundaries (@Transactional) for operations that update multiple related records together.
- Add caching (e.g., Spring Cache) for frequently requested summary statistics to reduce repeated computation.
- Write clear, well-documented service methods that the controller layer can call without duplicating logic.

Expected Deliverable: **Service Layer —** A robust backend service layer implementing groundwater-specific business rules — status classification, trend analysis, and stale-data detection — ready to support the dashboard and reporting features.

Technology & Purpose

Technology	Purpose
Spring Boot (Service layer)	Hosting business logic separate from controllers and repositories
Spring Data JPA	Querying data needed for aggregation and trend logic

Java Streams API	Performing in-memory calculations and aggregations efficiently
Spring Cache	Caching frequently computed summary statistics
Lombok	Reducing boilerplate in service classes and DTOs

June 26 — API Testing using Postman

Activity: On June 26, all backend REST APIs developed so far for the Real Time Groundwater Resource Evaluation System are tested using Postman to verify correctness, security, and reliability before frontend integration.

What is API Testing using Postman?

API Testing using Postman involves manually and semi-automatically sending HTTP requests to each backend endpoint and verifying that the responses match expected status codes, payloads, and error handling behavior. This is a critical checkpoint before connecting the React frontend, since any inconsistency in the API contract would otherwise surface as confusing frontend bugs.

Testing covers authentication flows, CRUD operations for stations and readings, role-based access restrictions, and edge cases such as invalid input or missing tokens.

Key Tasks Involved:

- Create a Postman collection covering all endpoints — authentication, station CRUD, reading CRUD, and dashboard summary APIs.
- Test successful login and registration flows, verifying that valid JWT tokens are returned and structured correctly.
- Test protected endpoints with and without a valid token to confirm role-based access control works as expected.
- Test Create and Update operations with valid and invalid payloads to confirm validation rules are enforced.
- Test Delete operations and confirm proper handling of attempts to delete non-existent or already-deleted records.
- Verify pagination, sorting, and filtering query parameters return correctly structured and accurate results.
- Use Postman environment variables to manage base URLs and tokens across development and testing environments.
- Document all test cases and results, noting any defects for resolution before frontend integration begins.

Expected Deliverable: **API Testing Report —** A documented Postman test report confirming that all backend APIs — authentication, CRUD, and role-based access — function correctly and consistently, ready for frontend integration.

Technology & Purpose

Technology	Purpose
Postman	Sending requests and validating API responses

Postman Collections & Environments	Organizing and reusing test requests across endpoints
Spring Boot (Backend under test)	The running application whose APIs are validated
JWT	Verifying token-based authentication during testing
Newman (optional)	Command-line execution of Postman collections for repeatable testing

June 27 — Frontend & Backend Integration

Activity: On June 27, the React frontend is integrated with the Spring Boot backend for the Real Time Groundwater Resource Evaluation System, connecting all previously built UI modules to live, real backend data.

What is Frontend & Backend Integration?

Frontend & Backend Integration is the process of replacing mock or static data in the React application with live calls to the Spring Boot REST APIs, so that login, dashboard, CRUD forms, and data tables all operate on real groundwater data flowing through the full application stack.

This stage often surfaces mismatches between what the frontend expects and what the backend returns, making careful end-to-end verification essential before the application can be considered a working whole.

Key Tasks Involved:

- Configure Axios base URL and interceptors to attach the JWT token automatically to every authenticated request.
- Connect the Login and Registration pages to the authentication APIs and handle token storage securely.
- Connect the Dashboard to live summary and station-data APIs, replacing any placeholder/mock data.
- Connect all CRUD forms (Add/Edit/Delete Station and Reading) to their respective backend endpoints.
- Connect the data table and search/filter UI to backend pagination, sorting, and filtering query parameters.
- Implement global error handling on the frontend for failed API calls, expired tokens, and network errors.
- Verify CORS configuration allows smooth communication between the frontend and backend ports/domains.
- Perform end-to-end walkthroughs of every user flow — login, view dashboard, add/edit/delete records, search/filter — to confirm correct behavior.

Expected Deliverable: Working Application — A fully integrated, end-to-end working application where the React frontend operates entirely on live data from the Spring Boot backend, covering authentication, dashboard, CRUD, and search/filter features.

Technology & Purpose

Technology	Purpose
React.js	Frontend application consuming live backend data

Axios	Making authenticated HTTP requests to the backend
Spring Boot (Backend)	Serving real data through REST APIs
JWT	Maintaining authenticated sessions across the integrated app
CORS Configuration	Enabling secure cross-origin communication between frontend and backend
Chrome DevTools	Debugging network requests during integration

June 28 — Bug Fixing & Validation

Activity: On June 28, the issues identified during frontend testing and integration are resolved for the Real Time Groundwater Resource Evaluation System, and the application is validated end-to-end for stability and correctness.

What is Bug Fixing & Validation?

Bug Fixing & Validation is the phase where defects logged during earlier testing and integration — incorrect data display, broken validations, UI inconsistencies, or backend errors — are systematically resolved. Validation then re-confirms that each fix works correctly and has not introduced new issues elsewhere in the application.

For a groundwater monitoring system, particular attention is given to data accuracy (water-level values, statuses, and trends) and to role-based access, since incorrect permissions or miscalculated thresholds could mislead field decisions.

Key Tasks Involved:

- Review and prioritize the bug list compiled from frontend testing, integration testing, and Postman API testing.
- Fix data-binding and rendering issues in the dashboard, tables, and forms identified during testing.
- Resolve validation mismatches between frontend form rules and backend Bean Validation constraints.
- Fix role-based access issues, ensuring Admin, Field Engineer, and Viewer permissions behave exactly as designed.
- Re-test all CRUD operations after fixes to confirm no regressions were introduced.
- Verify correctness of groundwater status calculations (Normal/Warning/Critical) against sample datasets.
- Re-run cross-browser and cross-device checks to confirm UI consistency after fixes.
- Perform a final smoke test of the complete application flow from login through to report/export functionality.

Expected Deliverable: Tested Project — A stable, validated application with previously identified bugs resolved and confirmed through re-testing, ready for documentation and final demonstration.

Technology & Purpose

Technology	Purpose
React.js	Frontend fixes for UI and data-binding issues

Spring Boot	Backend fixes for validation and business-logic errors
Jest / React Testing Library	Re-running automated frontend tests after fixes
Postman	Re-verifying API behavior after backend fixes
Chrome DevTools	Final cross-browser/device validation

June 29 — Documentation & PPT Preparation

Activity: On June 29, the complete project documentation and presentation slides are prepared for the Real Time Groundwater Resource Evaluation System, summarizing the system's design, development process, and outcomes.

What is Documentation & PPT Preparation?

Documentation & PPT Preparation involves consolidating the entire project's planning, design, and development work into a clear written report and an accompanying presentation deck. This includes problem statement, objectives, system architecture, technology stack, module-wise implementation details, screenshots, and testing outcomes.

A well-prepared report and presentation make it possible for evaluators, mentors, or future developers to quickly understand what the system does, how it was built, and what value it delivers for real-time groundwater monitoring.

Key Tasks Involved:

- Compile the project report covering introduction, problem statement, objectives, scope, and literature/technology overview.
- Document the system architecture, including frontend, backend, and database layers with diagrams (ER diagram, use case diagram, architecture diagram).
- Document each module — authentication, dashboard, CRUD, search/filter — with screenshots and a brief explanation of functionality.
- Summarize testing methodology and results, referencing the frontend testing report and API testing report.
- List the complete technology stack used across frontend, backend, and database layers with justification for choices.
- Prepare a PowerPoint presentation summarizing the problem, solution, architecture, key screens, and outcomes for the final demo.
- Proofread the report and presentation for clarity, consistent formatting, and correct technical terminology.
- Prepare a brief project abstract/summary suitable for inclusion in academic or organizational records.

Expected Deliverable: **Project Report & PPT —** A complete, well-structured project report and presentation deck summarizing the Real Time Groundwater Resource Evaluation System's design, development, testing, and outcomes — ready for final review.

Technology & Purpose

Technology	Purpose
Microsoft Word / Google Docs	Drafting and formatting the written project report
Microsoft PowerPoint / Google Slides	Preparing the final presentation deck
draw.io / Lucidchart	Creating ER diagrams, use case diagrams, and architecture diagrams
Screenshots from React App	Illustrating module functionality in the report

June 30 — Final Demo & Submission

Activity: On June 30, the Real Time Groundwater Resource Evaluation System is demonstrated in its complete, integrated form, and the final project — including source code, documentation, and presentation — is submitted.

What is Final Demo & Submission?

The Final Demo & Submission marks the conclusion of the development cycle, where the fully working application — covering authentication, dashboard, CRUD operations, search/filter features, and live DWLR data visualization — is presented end-to-end to evaluators or stakeholders.

Submission involves packaging all project artifacts — source code (frontend and backend), the project report, the presentation deck, and any supporting diagrams — into a final, organized deliverable.

Key Tasks Involved:

- Prepare a clean, working build of the application (frontend and backend) for the live demonstration.
- Walk through the complete user journey during the demo — login, dashboard overview, adding/editing station and reading data, search/filter, and status indicators.
- Highlight the role-based access control by briefly demonstrating Admin, Field Engineer, and Viewer views.
- Address questions from evaluators regarding architecture choices, business logic, and testing approach.
- Push the final, stable version of the source code to the version control repository (Git) with a clear commit history.
- Package the final project report and presentation deck alongside the source code for submission.
- Verify that all expected deliverables from the entire project timeline are complete and included in the submission.
- Conduct a final review/retrospective noting potential future enhancements (e.g., predictive analytics, mobile app, SMS alerts for critical levels).

Expected Deliverable: Final Project Submission — The complete Real Time Groundwater Resource Evaluation System — fully functional frontend and backend, tested and documented — successfully demonstrated and submitted along with its source code, report, and presentation.

Technology & Purpose

Technology	Purpose
React.js	Frontend application demonstrated during the final demo
Spring Boot	Backend APIs powering the live demonstration
MySQL / PostgreSQL	Database holding the final demo dataset
Git / GitHub	Version control and final source code submission
Microsoft PowerPoint	Presentation deck used during the final demo